Brain OS — Manual de arquitectura para un sistema personal de conocimiento operativo con IA

Guía completa para construir un ecosistema donde Claude (u otro modelo) funcione como colaborador operativo

Versión: 1.6 genérica — 2026
Propósito: Este documento es el punto de partida para que cualquier persona construya su propio sistema de inteligencia operativa personal. Es un manual maestro que describe arquitectura, identidad del asistente, memoria, herramientas y flujos de trabajo. Puede entregarse directamente a Claude (vía `claude.md`, sistema RAG o contexto inicial) para que entienda el sistema.

Sobre el nombre: El documento usa "Brain OS" como nombre genérico del sistema. En la práctica cada persona le pone un nombre propio — por ejemplo `[TuInicial]-Brain`, `SegundoCerebro`, `OS-Personal`, etc. Reemplazá "Brain OS" por el nombre que prefieras al implementarlo.

PARTE 0 — EL PROBLEMA QUE RESUELVE BRAIN OS

0.1 El problema

Si trabajás con múltiples proyectos o clientes en paralelo, tu contexto operativo vive disperso: correos en Gmail, documentos en Drive, notas en Obsidian o Notion, acuerdos en tu cabeza. Cada lunes arrancás reconstruyendo qué quedó pendiente. Cada vez que alguien te pregunta el estado de un proyecto, tardás diez minutos en armar una respuesta que debería estar lista en treinta segundos.

El problema no es la cantidad de información. Es que esa información no está conectada, no está en un formato que un asistente de IA pueda usar directamente, y no hay un sistema que la mantenga actualizada sin que vos lo hagás manualmente.

Brain OS resuelve eso. Es una arquitectura que convierte a un modelo de lenguaje en un colaborador operativo real: uno que conoce el estado de tus proyectos, recuerda tus preferencias, procesa documentos sin que tengas que decirle cómo hacerlo cada vez, y produce archivos listos — no respuestas para copiar.

El resultado concreto: en lugar de buscar información dispersa, abrís un HTML generado automáticamente. En lugar de redactar un correo desde cero, aprobás un borrador que ya tiene el tono y contexto del cliente. En lugar de recordar qué acordaste en la reunión del martes, el sistema ya lo registró, lo vinculó al proyecto, y lo marcó como pendiente si corresponde.

PARTE 1 — PERFIL DEL USUARIO Y CÓMO TRABAJA

1.1 ¿Para quién es este manual?

Este manual sirve a cualquier persona que cumpla al menos tres de estas características:

- **Trabaja por proyectos o clientes múltiples** en paralelo (consultores, freelancers, abogados independientes, directores de proyectos, académicos con múltiples líneas de investigación, creadores de contenido con varios formatos).
- **Genera y recibe documentación viva**: transcripciones, propuestas, actas, correos, contratos que se actualizan constantemente.
- **Acumula conocimiento de referencia** que necesita accesible pero no cambia mucho (papers, frameworks, regulación, bibliografía, materiales de curso).
- **Opera desde múltiples dispositivos** y necesita continuidad entre computadora, teléfono y trabajo delegado.
- **Prefiere outputs sobre conversaciones**: no quiere texto para copiar, quiere archivos listos, correos redactados, informes generados.

Si tu trabajo se puede describir como "manejo volumen alto de información sobre proyectos específicos y produzco entregables", este manual es para vos.

1.1.1 Qué NO es Brain OS

Brain OS es un **sistema operativo de trabajo**. Está diseñado para producir outputs verificables vinculados a proyectos, clientes, entregables y oportunidades comerciales. Su lógica gira alrededor de ciclos de vida observables — propuesta, ejecución, entrega, archivo.

Lo que Brain OS no es:

- **No es un sistema de journaling reflexivo.** Si querés llevar un diario personal sobre tu vida interna, dudas, emociones o procesos psicológicos, usá un vault separado, un cuaderno, o herramientas pensadas para eso. Mezclar journaling íntimo con el sistema donde Claude tiene acceso a tus proyectos y contactos comerciales borra una frontera que conviene mantener clara.
- **No es un sustituto de apoyo terapéutico o psicológico.** Claude en este sistema es un colaborador operativo, no un interlocutor para procesar emociones. Para eso están los profesionales de la salud mental, las redes de apoyo personales, y tu propio criterio.

- **No es una herramienta de gestión de salud, hábitos personales o finanzas íntimas.** Esos dominios tienen herramientas mejores y, más importante, no deberían vivir en el mismo espacio donde tu asistente toma decisiones sobre tus clientes.

Mantener este scope claro previene el *scope creep* que erosiona la utilidad del sistema y la claridad sobre qué rol cumple Claude en tu vida.

1.2 Principios operativos que asume el sistema

1. **Trabajo estructurado por proyectos con ciclo de vida definido.** Cada proyecto tiene propuesta → contrato → ejecución → entregables → facturación → cierre.
2. **Información viva separada de información estática.** Lo que cambia vive en bitácoras cronológicas. Lo que no cambia vive en carpetas de insumos o biblioteca.
3. **Claude escribe, vos revisás.** El asistente mantiene el vault. Vos rara vez editás archivos directamente.
4. **Outcomes sobre chat.** El sistema produce archivos listos, no respuestas para copiar.
5. **Memoria persistente.** El sistema recuerda correcciones, preferencias, decisiones.
6. **Doble vía humano-IA.** Los outputs importantes se generan como artefactos navegables (HTML), no como muros de texto que solo el agente lee.

1.3 Estilo de comunicación recomendado

- Directo y sin relleno.
- Listas cuando hay múltiples items; prosa cuando es análisis.
- Un idioma principal para documentación; técnica en inglés si conviene.
- Contexto antes de recomendaciones.
- No repetir información que ya está en el vault.

PARTE 2 — ARQUITECTURA DEL SISTEMA

2.1 Filosofía base (modelo compilador de Andrej Karpathy)

```
None
raw/ (fuentes brutas)
  → Compiler (LLM procesa y estructura)
    → wiki/ (conocimiento compilado, navegable)
      → runtime (Claude consulta y actúa)
        → outputs/ (archivos listos — Markdown o HTML según función)
```

Principio central: Claude escribe y mantiene la base de conocimiento. El usuario rara vez toca los archivos directamente.

Principio operativo clave: Compilar significa decirle a Claude "procesá esto". Claude es el compilador.

2.2 Markdown vs. HTML como formato de output *(nuevo en v1.4)*

Brain OS usa dos formatos según la función del archivo:

Markdown (`.md`) para todo lo que es fuente de verdad editable: notas activas, logs, overviews, memory, claude.md, contactos del CRM, contenido de biblioteca.

HTML (`.html`) para lo que el usuario lee como dashboard, no como texto: estados de proyecto, briefs diarios, reportes de lint, paneles de pendientes, timelines, cierres de proyecto.

Por qué HTML para esos outputs: un `.md` largo con tablas y listas anidadas es difícil de escanear. Un `.html` autocontenido con secciones colapsables, código de color por estado, gráficos SVG inline, y links navegables se lee en segundos. Y como sigue siendo un archivo de texto que vive en el vault, Claude lo puede leer también — el HTML se convierte en contexto compartido entre vos y Claude, no solo un artefacto descartable.

Regla de decisión:

- ¿El archivo es la fuente de verdad de algo que el usuario o Claude editan? → **Markdown**
- ¿El archivo es una vista derivada de otra información del vault, generada para ser leída? → **HTML**

Los HTML pueden regenerarse en cualquier momento desde su fuente. No son inmutables — son vistas compiladas.

Convención de nomenclatura para HTML

```
None
[tipo-funcional]_[descripción]_[proyecto-o-ámbito].html
```

Tipos canónicos:

Tipo	**Para qué se usa**
`dashboard`	Vista de estado (proyecto, finanzas, semana)
`panel-control`	Vista interactiva con controles o filtros
`brief`	Resumen ejecutivo (diario, semanal, de cierre)
`reporte`	Documento estructurado de salida formal
`timeline`	Cronología visual
`comparativo`	Side-by-side de opciones o versiones
`mapa`	Diagrama o flujo de proceso
`cierre`	HTML de archivo final de proyecto

Ejemplos:

- `dashboard_proyecto-acme_estado-actual.html`
- `panel-control_pendientes_acme.html`
- `brief_diario_2026-05-09.html`
- `cierre_proyecto-fundepos_v1.html`
- `reporte_vault-lint_2026-05-08.html`
- `timeline_costa-rica-tech-week_2026.html`

Dónde viven los HTML:

- HTML de un proyecto → `02-Proyectos/[Cliente]/outputs-ai/`
- HTML diarios (briefs, daily summary) → `03-Daily-logs/`
- HTML del propio sistema (vault-lint, memory-review) → `00-Sistema/reportes/`

2.3 Las cuatro capas funcionales

Capa A — Ingesta bruta (`raw/`)

Cuatro mecanismos de ingesta:

Mecanismo 1 — Arrastre manual a carpeta El usuario copia archivos físicos (PDFs, contratos, transcripciones) a `raw/[proyecto]/`. Claude los procesa cuando se le indica.

Mecanismo 2 — Conectores MCP Para Gmail y Google Drive, Claude jala la información directamente cuando se le pide.

Mecanismo 3 — Obsidian Web Clipper Artículos y papers de internet van directamente a `raw/biblioteca/` como Markdown.

Mecanismo 4 — Correo del asistente como canal de ingesta pasiva La cuenta `asistente@[tu-dominio].com` solo recibe, nunca envía. Recibe transcripciones automáticas de Google Meet, confirmaciones de servicios, y documentos enviados por terceros.

Cuadro resumen — por dónde entra cada cosa

Fuente	**Cómo entra**	**Quién compila**
PDF/docs recibidos	Usuario los copia a `raw/[proyecto]/`	Claude cuando se le indica
Correos Gmail (usuario)	MCP los jala directo	Claude a pedido
Drive del cliente	MCP los jala directo	Claude a pedido
Artículos web	Web Clipper → `raw/biblioteca/`	Skill `library-compiler`
Transcripciones Meet	Auto → correo asistente → MCP	Skill `meet-processor`
Docs de referencia	Usuario copia a `raw/vector-inbox/`	Skill `vector-ingest` → Pinecone
Sesiones de trabajo	Hooks automáticos (Fase 2)	Flush diario automático

Capa B — Compilador

Skills que transforman información bruta en conocimiento estructurado. Ver Parte 6 para detalle completo.

Capa C — Vault (Obsidian + GitHub + Pinecone)

Ver sección 2.4 para la estructura completa.

Capa D — Runtime (Query + Acción)

- **Claude Cowork** — tareas que producen archivos, skills, scheduled tasks.
- **Claude.ai + MCP** — consultas diarias, acceso a Gmail/Drive/Calendar.
- **Claude Code** — automatización avanzada, hooks, flush automático (Fase 2).
- **Dispatch (móvil)** — delegar tareas desde el teléfono.

2.4 Estructura del vault y territorio de acceso

```
None
Brain-OS/ (vault raíz)
│
│  ═══════════════════════════════════
│  TERRITORIO CLAUDE — lee y escribe
│  ═══════════════════════════════════
│
├── 00-Sistema/
│   ├── claude.md              ← instrucciones maestras del sistema
│   ├── index.md               ← mapa de navegación del vault
│   ├── projects.base          ← dashboard de estado de proyectos
│   ├── memory.md              ← preferencias y correcciones aprendidas
│   ├── entity-registry.md     ← registro canónico de entidades
│   ├── skills/                ← definición de skills + feedback por skill
│   │   └── [nombre-skill]/
│   │       ├── definicion.md
│   │       └── feedback.md    ← log de ajustes pedidos (Super skills)
│   ├── reportes/              ← HTML del propio sistema (vault-lint,
memory-review)
│   └── prompts-maestros/
│
├── 01-Biblioteca/
│   ├── frameworks/
│   ├── papers/
│   ├── regulacion/
│   ├── cursos/
│   └── index-tematico.md
│
```

```
├── 02-Proyectos/
│   └── [Cliente]/              ← ver estructura detallada en sección 2.5
│
├── 03-Daily-logs/              ← contiene .md y .html (briefs, dailies)
│
├── 04-Inbox-Claude/
│   └── sin-proyecto/
│
├── 05-CRM/
│   └── [Nombre-Contacto].md
│
├── 06-Archivo/
│   ├── proyectos-cerrados/
│   └── contactos-inactivos/
│
│  ═══════════════════════════════════════════════════════
│  TERRITORIO USUARIO — Claude solo accede si se le pide
│  ═══════════════════════════════════════════════════════
│
├── 07-Finanzas/
│   ├── pagos-por-realizar/
│   └── cuentas-por-cobrar/
│
│  ═══════════════════════════════════════════════════════
│  TERRITORIO COMPARTIDO
│  ═══════════════════════════════════════════════════════
│
├── 08-Funnel/
│   └── funnel.md
│
└── raw/
    ├── [Proyecto]/
    │   └── procesados/
    ├── biblioteca/
    │   └── procesados/
    └── vector-inbox/
        └── procesados/
```

Regla de territorio (no negociable):

- Claude **nunca** abre, lee ni modifica archivos en `07-Finanzas/` salvo instrucción explícita del usuario en esa sesión.
- Claude **sí puede** leer `07-Finanzas/` cuando el usuario lo pide directamente.
- El skill `vault-lint` verifica esta regla semanalmente.

2.5 Estructura interna de cada proyecto

```
None
02-Proyectos/[Cliente]/
├── overview.md          ← perfil, alcance, contactos clave, estado
├── log.md               ← bitácora cronológica parseable
├── claude-project.md    ← preferencias específicas del proyecto (opcional)
├── insumos/
├── entregables/
├── financiero/
└── outputs-ai/          ← drafts en .md y dashboards en .html
```

Project Operating Manual — `claude-project.md` *(nuevo en v1.4)*

Archivo opcional. Solo se crea cuando un proyecto tiene preferencias específicas que se stackean sobre las globales. Si no existe, Claude usa solo `claude.md` global y `overview.md`.

Ejemplos de cuándo crearlo:

- "Mocapán prefiere comunicación por WhatsApp, no correo."
- "Para FUNDEPOS evitar lenguaje técnico en presentaciones."
- "Acme Corp requiere que toda comunicación pase por [contacto] antes de enviarse."
- "Este cliente firma con membrete específico — usar siempre la plantilla."

```
None
---
proyecto: [Nombre]
creado: YYYY-MM-DD
ultima-actualizacion: YYYY-MM-DD
---

## Preferencias específicas de este proyecto
- [Regla 1]
- [Regla 2]

## Tono y estilo para este cliente
[Cualquier consideración particular]

## Convenciones internas del cliente
- [Términos que usan distinto al estándar]
- [Formatos preferidos]

## Stakeholders y dinámicas
- [Quién aprueba qué]
```

```
- [Sensibilidades a evitar]
```

Stacking de instrucciones: cuando Claude trabaja en un proyecto, lee en este orden — `claude.md` global → `claude-project.md` del proyecto (si existe) → `overview.md` → `log.md` reciente. Lo más específico gana en caso de conflicto.

Formato de `log.md` con prefijo parseable

```
None
## [YYYY-MM-DD] reunión | Kickoff con cliente
**Tipo:** reunión | correo | llamada | entregable | transcripción-meet
**Resumen:** [1-2 oraciones]
**Acuerdos:**
**Pendientes del usuario:**
**Próxima acción:** [qué pasa next y cuándo]
**Fuente:** [reunión directa | correo de X | transcripción Meet YYYY-MM-DD]
```

Tipos válidos: `reunión` | `correo` | `llamada` | `entregable` | `ingest` | `transcripción-meet` | `decisión` | `alerta`

PARTE 3 — CRM Y FUNNEL

3.1 CRM — un archivo por contacto

Cada contacto vive en `05-CRM/[Nombre-Apellido].md`. El YAML es la fuente de verdad. Las notas cronológicas debajo son el historial.

```
None
---
nombre: [Nombre completo]
empresa: [Organización]
rol: [Cargo]
correo: [email]
relacion: cliente | prospecto | aliado | institucional | personal
estado: activo | inactivo | archivado
proyectos-asociados:
  - "[[02-Proyectos/Nombre-Proyecto]]"
ultima-actualizacion: YYYY-MM-DD
```

```
---

## Notas de relación

### [YYYY-MM-DD]
[Entrada más reciente primero]
```

3.2 Funnel — tabla dinámica de oportunidades

`08-Funnel/funnel.md`:

```
None
| Oportunidad | Contacto | Monto | Fecha | Estado | Next Step |
|-------------|----------|-------|-------|--------|-----------|
| [[02-Proyectos/X]] | [[05-CRM/Nombre]] | $X,XXX | YYYY-MM-DD | prospecto |
[acción] |
```

Estados válidos: `prospecto` | `propuesta-enviada` | `negociación` | `cerrado-ganado` | `cerrado-perdido` | `en-pausa`

Para vista navegable, Claude puede generar `dashboard_funnel_global.html` con filtros por estado, monto y fecha. Se regenera bajo demanda desde `funnel.md`.

PARTE 4 — IDENTIDAD DE CLAUDE EN ESTE SISTEMA

4.1 Qué es Claude en este ecosistema

Claude es un **colaborador con infraestructura propia**:

- **Correo propio** (`asistente@[dominio].com`): solo recibe.
- **Google Drive de trabajo**: shared drives del usuario.
- **Slack por canales de acción**: bitácora de trabajo por tipo.
- **Carpeta local**: `/Brain-OS/`.
- **GitHub**: repositorio privado del vault.
- **Obsidian CLI**: lee y escribe el vault programáticamente.
- **Pinecone**: corpus de referencia.

4.2 Reglas de identidad de Claude (10 reglas)

1. Claude escribe el vault, el usuario lo revisa.
2. Claude entrega archivos, no respuestas de chat.
3. Claude pregunta antes de borrar, sobreescribir o renombrar.
4. Claude aprende de cada corrección → actualiza `memory.md` (general) o `feedback.md` del skill (específico).
5. Claude mantiene `index.md` actualizado.
6. Claude usa estado en cada output Markdown (`estado:` en frontmatter).
7. Claude mueve `raw/` a `raw/[X]/procesados/` después de destilar.
8. Claude reporta en Slack al terminar cualquier procesamiento.
9. Claude respeta el territorio del usuario — nunca accede a `07-Finanzas/` salvo instrucción explícita.
10. Claude verifica el Entity Registry antes de crear un contacto, proyecto o concepto nuevo.

4.3 Stacking de instrucciones por nivel

Cuando Claude opera, lee instrucciones en este orden — lo más específico gana:

```
None
1. 00-Sistema/claude.md           ← constitución del sistema
2. 02-Proyectos/[X]/claude-project.md  ← preferencias del proyecto (si existe)
3. 02-Proyectos/[X]/overview.md   ← contexto del proyecto
```

```
4. 02-Proyectos/[X]/log.md        ← entradas recientes
5. 00-Sistema/memory.md           ← preferencias globales aprendidas
6. 00-Sistema/skills/[skill]/feedback.md  ← ajustes específicos al skill activo
```

4.4 Slack como bitácora de acciones

Canal	Qué reporta Claude ahí
`#brain-archivos`	Crea, mueve, renombra o archiva un documento
`#brain-procesados`	Termina de procesar `raw/`
`#brain-transcripciones`	Procesa una transcripción de Meet
`#brain-alertas`	Pendientes detectados, alertas
`#brain-daily`	Brief del día y flush de cierre

Formato estándar:

```
None
[TIPO] [Proyecto/Área] — [descripción breve]
Archivos: [lista o link a HTML si aplica]
Pendientes detectados: [lista | ninguno]
```

Tipos: `[PROCESADO]` `[TRANSCRIPCIÓN]` `[ARCHIVADO]` `[ALERTA]` `[BRIEF]` `[FLUSH]` `[VAULT-LINT]` `[CRM]` `[FUNNEL]` `[DASHBOARD]`

PARTE 5 — ENTITY REGISTRY

5.1 El problema que resuelve

Sin un registro central, el vault termina con duplicados. El Entity Registry previene esto con una fuente de verdad canónica.

5.2 Estructura

`00-Sistema/entity-registry.md`:

```
None
## Personas
| Nombre canónico | Aliases | Archivo CRM | Notas |

## Organizaciones
| Nombre canónico | Aliases | Proyectos asociados | Notas |

## Conceptos y marcos
| Nombre canónico | Aliases | Archivos relacionados | Notas |

## Proyectos
| Nombre canónico | Aliases | Carpeta | Estado |
```

5.3 Reglas

1. Antes de crear cualquier entidad, leer el registry.
2. Si ya existe: usar el nombre canónico y crear el backlink.
3. Si no existe: agregar primero al registry, luego crear el archivo.
4. Claude nunca elimina entradas — solo las marca como archivadas.
5. El skill `vault-lint` verifica que no haya archivos sin entrada en el registry.

PARTE 6 — SISTEMA RAG Y MEMORIA LONG-TERM

6.1 Arquitectura de memoria — dos capas

Índices de texto en Obsidian:

```
None
00-Sistema/index.md
01-Biblioteca/index-tematico.md
02-Proyectos/[cliente]/overview.md
03-Daily-logs/index-logs.md
05-CRM/index-crm.md
```

Pinecone (corpus de referencia):

Regla: ¿ciclo de vida activo? → Obsidian. ¿Terminado y solo de consulta? → Pinecone.

Qué va a Pinecone: papers, regulación, libros, transcripciones históricas, escritos publicados, entregables finales, materiales de curso.

Metadatos obligatorios:

```
JSON
{
  "tipo": "paper | regulacion | libro | transcripcion | entregable | curso",
  "proyecto": "[cliente o 'biblioteca']",
  "fecha": "YYYY-MM-DD",
  "titulo": "[nombre del documento]",
  "fuente": "[URL o ruta en raw/procesados/]"
}
```

6.2 Tipos de claims en notas de biblioteca

Para notas en `01-Biblioteca/` que mezclan información de fuentes con análisis propio, Claude usa cuatro tipos de claims como callouts de Obsidian:

```
None
> [!source] Cita o dato directamente de la fuente
> "El artículo 5 establece que..." — Reglamento UE 2024/1689, p. 23

> [!analysis] Inferencia propia basada en fuentes citadas
> La combinación del artículo 5 con el considerando 47 sugiere que...
> **Fuentes base:** [[regulacion/eu-ai-act]]

> [!unverified] Afirmación sin fuente verificada aún
> Pendiente de confirmar con fuente primaria.

> [!gap] Información que debería existir pero no está en el vault
> No hay documentación sobre [tema].
```

`vault-lint` verifica que notas en `01-Biblioteca/` con más de 500 palabras tengan al menos un claim tipado.

6.3 Memoria de corto plazo — Daily logs

```
None
## [YYYY-MM-DD] sesión | 09:30-10:15
**Proyecto:** Acme Corp
**Tarea:** Revisión de gaps en propuesta v2
**Decisiones:** Se prioriza [X] antes que [Y]
**Pendientes:** Enviar análisis comparativo antes del viernes
```

6.4 Memoria de largo plazo — Daily flush

Una vez al día, Claude promueve daily logs a la wiki:

- Decisiones → `overview.md` del proyecto.
- Lecciones → `00-Sistema/memory.md`.
- Conexiones entre proyectos → backlinks en Obsidian.
- Items sin proyecto → `04-Inbox-Claude/`.
- Movimientos comerciales → `08-Funnel/funnel.md`.
- Nuevos contactos → `05-CRM/` y Entity Registry.

Al final del flush, Claude regenera los HTML afectados (dashboards de proyectos con cambios, brief del día, etc).

PARTE 7 — HERRAMIENTAS Y CONECTORES

7.1 Stack completo del sistema

Herramienta	**Rol**	**Fase**
Obsidian	IDE del vault, visualización	1
GitHub (privado)	Version control, backup	1
Plugin Git (Obsidian)	Auto-sync cada 1 min	1
Obsidian Web Clipper	Captura artículos web	1
Claude.ai + MCP	Chat diario, consultas	1
Gmail MCP (cuenta usuario)	Correos por proyecto	1
Gmail MCP (cuenta asistente)	Ingesta pasiva: Meet, docs	1
Drive MCP (shared drives)	Docs de clientes	1
Calendar MCP	Agenda, eventos	1
Pinecone	Vector search — corpus referencia	1
Slack (canales por tipo)	Bitácora de acciones	1
Claude Cowork	Agente local, skills	1-2
Dispatch (móvil)	Tareas remotas	2
Claude Code	Hooks, automatización	2
Obsidian CLI	Cowork opera el vault	2
Browser local	Visualización de HTML del vault	1

7.2 Agnosticismo al runtime

Brain OS no depende de un cliente o IDE específico. El sistema es un conjunto de archivos en un vault Obsidian, conectores MCP, una base vectorial, y una serie de skills definidos en Markdown. Cualquier interfaz que pueda leer y escribir esos archivos y ejecutar esos conectores funciona.

Opciones de runtime conocidas al día de hoy:

Runtime	**Cuándo conviene**
Claude Cowork	Por defecto. El más integrado con MCP y skills.
Claude Code	Para Fase 3 y hooks. Mejor para automatización avanzada.
OpenAI Codex	Si ya estás dentro del ecosistema ChatGPT. Funciona igual; solo cambian los conectores.
Dispatch	Para delegar desde móvil sin estar frente a la computadora.

El manual usa el lenguaje de Claude Cowork porque es el ecosistema donde el sistema fue diseñado y probado. **Nota sobre portabilidad:** aunque la arquitectura es agnóstica en teoría — vault, MCP, skills en Markdown —, una guía detallada de lift-and-shift a otros modelos (equivalencias de hooks, conectores alternativos, diferencias en cómo cada modelo usa el contexto) es trabajo pendiente para futuras versiones. Por ahora, **se recomienda implementar Brain OS con Claude**. Si en algún momento cambiás de modelo, el vault y los skills son el activo portable; lo que varía son los conectores y la sintaxis de automatización.

7.3 Mapa de deployment de skills *(nuevo en v1.5)*

No todos los skills automatizados se ejecutan del mismo modo. Cada uno tiene un método de deployment óptimo según sus requisitos técnicos:

Skill	**Método de deployment**	**Por qué**
`daily-brief`	Cloud routine (Anthropic)	Tiene que correr a las 7:30 AM aunque la máquina esté apagada

Skill	Método de deployment	Por qué
`flush-diario`	Cloud routine	Mismo motivo — corre 18:00 sin depender de la máquina
`meet-processor`	Cloud routine o loop manual	Procesa transcripciones al cierre del día
`vault-lint`	Local scheduled task	Requiere acceso CLI a Obsidian, vive en el repo local
`skill-tuning`	Local scheduled task	Lee y escribe en `00-Sistema/skills/`, mejor con repo abierto
`memory-review`	Local scheduled task	Mismo motivo — manipula `00-Sistema/memory.md`
`pinecone-audit`	Local scheduled task	Necesita comparar Pinecone con el índice local
`raw-processor`	A demanda + loop manual cuando hay backlog	No requiere schedule fijo
`library-compiler`	A demanda	Se dispara cuando el usuario decide procesar la biblioteca
`onboard-project`	A demanda	Lo activa el usuario al iniciar un proyecto nuevo

Diferencias clave

Cloud routines (corren en infraestructura de Anthropic):

- Funcionan con la máquina apagada y sin sesión abierta.
- Ideales para tareas con schedule fijo (briefs, flushes).
- Limitadas por plan (5/día Pro, 15/día Max, 25/día Team).
- No tienen acceso al CLI de Obsidian local — operan vía MCP.

Local scheduled tasks (corren en la máquina del usuario):

- Requieren la app de Claude (Cowork, Code, etc.) abierta y la máquina encendida.
- Si la máquina estuvo apagada, Cowork "pone al día" las tareas perdidas al volver — para evitarlo, pausarlas si vas a estar offline por días.

- Ideales para tareas que necesitan tocar el vault directamente (lint, tuning).

Loops de sesión (`/loop` en Cowork o Claude Code):

- Útiles para procesamiento por lotes en una jornada de trabajo.
- Mueren al cerrar la sesión.
- Expiran a los 7 días por diseño.
- Para loops largos que procesan muchos archivos, conviene un segundo loop más frecuente que ejecute `/clear` cada 30-60 minutos. Esto previene context rot durante procesamiento masivo (relevante sobre todo para `raw-processor` y `library-compiler` con backlogs grandes).

7.4 Guía de selección de modelos *(nuevo en v1.6)*

No todas las tareas de Brain OS requieren el mismo modelo. Usar el modelo correcto para cada tarea reduce costo, mejora velocidad, y en algunos casos mejora la calidad del output.

> ⚠️ **Esta sección se desactualiza rápido.** Las familias de modelos, sus capacidades y precios cambian con frecuencia. Revisá siempre la documentación oficial antes de tomar decisiones de implementación: platform.claude.com/docs/en/about-claude/models/overview

Familias de modelos disponibles (al momento de esta edición)

Modelo	Perfil	Contexto
Claude Fable 5	El más capaz. Razonamiento complejo, workflows largos.	1M tokens
Claude Opus 4.8	Alto nivel para ingeniería de software y tareas multi-paso.	1M tokens
Claude Sonnet 4.6	Balance de inteligencia, velocidad y costo. "Daily driver" recomendado.	1M tokens
Claude Haiku 4.5	El más rápido. Parsing de documentos, tareas de alto volumen.	—

Recomendación por tipo de tarea en Brain OS

Skill / Tarea	Modelo recomendado	Justificación
`daily-brief`	Sonnet	Síntesis de múltiples fuentes, output estructurado — buena relación velocidad/calidad
`flush-diario`	Sonnet	Volumen alto de procesamiento diario; calidad suficiente para promover logs
`estado-proyecto`	Sonnet	Análisis de contexto + generación de HTML navegable

Skill / Tarea	Modelo recomendado	Justificación
`draft-comunicacion`	Sonnet o Opus	Si el tono es crítico (propuestas, contratos), subir a Opus
`library-compiler`	Sonnet	Clasificación y síntesis de artículos — tarea repetitiva de volumen
`raw-processor`	Haiku o Sonnet	Extracción y clasificación estructurada — no requiere razonamiento complejo
`meet-processor`	Sonnet	Destilación de transcripciones — requiere juicio sobre qué es acuerdo vs. contexto
`vector-ingest`	Haiku	Fragmentación y extracción de metadatos — tarea mecánica de alto volumen
`vault-lint`	Sonnet	Requiere leer múltiples archivos y detectar patrones
`skill-tuning`	Opus o Fable	Análisis de patrones en feedback + propuesta de ajustes estructurales — alta carga cognitiva
Consultas directas en chat	Sonnet	Queries diarios, estado, preguntas puntuales
Diseño o revisión de arquitectura del sistema	Fable o Opus	Razonamiento sobre el sistema en su conjunto

Criterio de decisión rápido

- **¿La tarea procesa volumen alto de texto con lógica repetitiva?** → Haiku.
- **¿La tarea requiere síntesis, redacción o análisis de contexto?** → Sonnet.
- **¿La tarea requiere razonamiento complejo, juicio sobre el sistema, o el output es de alto impacto?** → Opus o Fable.

PARTE 8 — SKILLS Y FLUJOS DE TRABAJO

8.1 Dos tipos de skills *(nuevo en v1.4)*

Brain OS distingue dos tipos de skills con lógica de mantenimiento distinta:

Utility skills

Una sola tarea, sin memoria entre ejecuciones, sin feedback loop. Son herramientas que hacen algo concreto y terminan.

Ejemplos: `raw-processor`, `archive-project`, `vector-ingest`, `meet-processor`, `crm-update`.

Mantenimiento: revisar trimestralmente que el flujo siga siendo correcto. Sin feedback.md.

Super skills

Tareas complejas con feedback loop. Tienen memoria propia (su `feedback.md`), aprenden de las correcciones del usuario, y se auto-ajustan en la rutina mensual `skill-tuning`.

Ejemplos: `daily-brief`, `estado-proyecto`, `draft-comunicacion`, `library-compiler`, `flush-diario`.

Mantenimiento: rutina mensual `skill-tuning` analiza los `feedback.md` de cada Super skill y propone ajustes estructurales al prompt del skill.

Estructura de carpeta por skill

```
None
00-Sistema/skills/
├── raw-processor/
│   └── definicion.md           ← solo definición (Utility skill)
│
├── daily-brief/
│   ├── definicion.md           ← prompt y flujo del skill
│   └── feedback.md             ← log de ajustes pedidos (Super skill)
│
├── estado-proyecto/
│   ├── definicion.md
│   └── feedback.md
│
```

```
└── ...
```

Formato de `feedback.md`

```
None
# Feedback acumulado — [nombre-skill]
Última revisión: YYYY-MM-DD

## [YYYY-MM-DD] ajuste-pedido
**Output original generado:** [resumen breve o link al archivo]
**Qué pidió el usuario corregir:** [descripción]
**Patrón detectado:** [si Claude identifica un patrón repetido]

## [YYYY-MM-DD] ajuste-pedido
...
```

Cada vez que el usuario corrige un output de un Super skill, Claude registra la corrección aquí (no en `memory.md` general, que es para preferencias globales).

8.2 Skills prioritarios

Utility skills

`onboard-project`

Trigger: "Onboardá el proyecto [Cliente]" **Flujo:**

1. Lee Gmail con etiqueta del cliente (últimos 90 días).
2. Lee carpeta del cliente en Drive.
3. Crea estructura en `02-Proyectos/[Cliente]/` y `raw/[Cliente]/`.
4. Genera `overview.md` y `log.md`.
5. Verifica Entity Registry — crea o vincula contactos.
6. Actualiza `05-CRM/` con contactos nuevos.
7. Actualiza `08-Funnel/funnel.md` si aplica.
8. Genera `dashboard_proyecto-[cliente]_estado-actual.html` inicial.
9. Reporta en `#brain-procesados`.

`raw-processor`

Trigger: "Procesá lo nuevo en `raw/[proyecto]/`" **Flujo:**

1. Lista archivos no procesados.

2. Clasifica: estático → `insumos/` o `entregables/`; dinámico → `log.md`.
3. Mueve originales a `procesados/`.
4. Actualiza `overview.md` si hay cambios de estado.
5. Reporta en `#brain-procesados`.

`meet-processor`

Trigger: "Procesá la transcripción de la reunión de hoy" **Flujo:**

1. Lee correos nuevos en cuenta del asistente.
2. Identifica proyecto por título del evento o participantes.
3. Destila: acuerdos, decisiones, pendientes.
4. Crea entrada en `log.md` con prefijo `## [YYYY-MM-DD] transcripción-meet | Título`.
5. Actualiza CRM de los participantes.
6. Archiva el correo original.
7. Reporta en `#brain-transcripciones`.

`vector-ingest`

Trigger: "Ingestá los documentos nuevos en `raw/vector-inbox/`" **Flujo:**

1. Lista archivos no procesados.
2. Por cada archivo: extrae metadatos, fragmenta, genera embeddings, ingesta a Pinecone.
3. Mueve a `raw/vector-inbox/procesados/`.
4. Agrega entrada en `01-Biblioteca/index-tematico.md`.
5. Reporta en `#brain-archivos`.

`crm-update`

Trigger: "Actualizá el CRM de [Contacto]" o automático desde otros skills **Flujo:**

1. Verifica Entity Registry — usa nombre canónico.
2. Si el archivo no existe, lo crea.
3. Agrega entrada cronológica arriba con fecha.
4. Actualiza `ultima-actualizacion` y `proyectos-asociados`.
5. Reporta en `#brain-archivos`.

`archive-project`

Trigger: "Cerrá el proyecto [Cliente]" **Flujo:**

1. Genera `cierre_proyecto-[cliente]_v1.html` con resumen ejecutivo, cronología y entregables.
2. Actualiza `estado: cerrado` en `overview.md`.

3. Mueve la carpeta a `06-Archivo/proyectos-cerrados/`.
4. Actualiza Entity Registry.
5. Actualiza `projects.base` y `funnel.md`.
6. Reporta en `#brain-archivos`.

Super skills (con feedback loop)

`daily-brief` ⚙️

Trigger: Automático 7:30 AM **Flujo:**

1. Lee `projects.base`, Calendar, Gmail prioritario.
2. Revisa correo del asistente — transcripciones pendientes.
3. Revisa `funnel.md` — oportunidades con next step vencido.
4. Genera `brief_diario_YYYY-MM-DD.html` con timeline visual del día, pendientes críticos, follow-ups comerciales.
5. Reporta en `#brain-daily` con link al HTML.
6. Si el usuario pide ajustes al brief, registra en `skills/daily-brief/feedback.md`.

`estado-proyecto` ⚙️

Trigger: "¿Cuál es el estado de [Cliente]?" **Flujo:**

1. Lee `claude-project.md` (si existe), `overview.md` y últimas 5 entradas de `log.md`.
2. Consulta Gmail por correos sin respuesta (últimos 7 días).
3. Verifica estado en funnel.
4. Genera `dashboard_proyecto-[cliente]_estado-actual.html` con vista navegable.
5. Si el usuario pide ajustes, registra en `skills/estado-proyecto/feedback.md`.

`draft-comunicacion` ⚙️

Trigger: "Redactá [tipo] para [Cliente] sobre [tema]" **Flujo:**

1. Lee stacking completo: `claude.md` → `claude-project.md` → `overview.md` → CRM del contacto.
2. Si el tema requiere contexto de biblioteca, consulta Pinecone.
3. Redacta y guarda en `outputs-ai/` con `estado: borrador`.
4. Si el usuario corrige el draft, registra en `skills/draft-comunicacion/feedback.md`.

`library-compiler` ⚙️

Trigger: "Procesá los artículos nuevos en `raw/biblioteca/`" **Flujo:**

1. Lista archivos no procesados.
2. Corto → crea artículo temático en `01-Biblioteca/[categoría]/` con claims tipados.
3. Largo → deriva a `vector-ingest`.
4. Actualiza `index-tematico.md`.
5. Si el usuario corrige clasificaciones, registra en `skills/library-compiler/feedback.md`.

`flush-diario` ⚙️

Trigger: Final de jornada (manual Fase 1, automático Fase 2) **Flujo:**

1. Lee todos los daily logs del día.
2. Actualiza `log.md`, `overview.md`, `memory.md` por proyecto.
3. Actualiza CRM y funnel con movimientos detectados.
4. Actualiza Entity Registry.
5. Regenera HTML afectados (dashboards de proyectos con cambios, brief del día).
6. Actualiza `index.md`.
7. Reporta en `#brain-daily`.
8. Si el usuario corrige cómo se promovieron decisiones, registra en `skills/flush-diario/feedback.md`.

Skills de cierre y sistema

`session-audit` *(nuevo en v1.4)*

Trigger: `/session-audit` al cierre de cualquier sesión **Flujo:**

1. Escanea la conversación completa.
2. Identifica: preferencias nuevas del usuario, convenciones expresadas, correcciones repetidas.
3. Clasifica cada hallazgo:
    - Preferencia global → propone agregar a `memory.md`.
    - Preferencia de proyecto → propone agregar a `claude-project.md` del proyecto activo.
    - Corrección a un Super skill → propone agregar a `feedback.md` del skill correspondiente.
4. Muestra al usuario los hallazgos antes de guardar.
5. Reporta en `#brain-archivos` qué se actualizó.

`update-claude-md`

Trigger: "Actualizá tu contexto basándote en lo que trabajamos hoy" **Flujo:**

1. Lee conversación reciente y daily logs.
2. Actualiza secciones "Proyectos activos" y "Estado de aprendizaje" en `claude.md`.
3. Muestra qué cambió antes de guardar.

`vault-lint`

Trigger: Viernes al cierre (manual Fase 1, automático Fase 2) **Flujo:**

1. Escanea overview.md de proyectos activos — detecta los no actualizados en más de 7 días.
2. Verifica proyectos en `projects.base` con carpeta correspondiente.
3. Detecta archivos huérfanos.
4. Verifica entidades en `05-CRM/` y `02-Proyectos/` contra Entity Registry.
5. Verifica que Claude no haya accedido a `07-Finanzas/` sin instrucción explícita.
6. Verifica claims tipados en notas largas de biblioteca.
7. Genera `reporte_vault-lint_YYYY-MM-DD.html` con secciones colapsables por tipo de issue.
8. Reporta en `#brain-alertas` con link al HTML.

`skill-tuning` ⚙️ *(nuevo en v1.4)*

Trigger: Primer lunes del mes (manual Fase 1, automático Fase 2) **Flujo:**

1. Lee `feedback.md` de todos los Super skills.
2. Detecta patrones recurrentes (ej: "5 veces el usuario pidió que los briefs sean más cortos").
3. Para cada patrón con 3+ ocurrencias, propone un ajuste al `definicion.md` del skill.
4. Genera `reporte_skill-tuning_YYYY-MM.html` con propuestas, justificación y ejemplos.
5. Si el usuario aprueba un ajuste, modifica el `definicion.md` del skill correspondiente.
6. Marca como "incorporado" las entradas de feedback que dieron lugar al ajuste.
7. Reporta en `#brain-alertas`.

8.3 Flujo de trabajo diario típico

```
None
07:30  → daily-brief corre automáticamente
         Genera brief_diario_YYYY-MM-DD.html
         Reporta en #brain-daily con link

08:00  → Usuario abre el HTML del brief, asigna prioridades

Durante el día:
  → Usuario deposita archivos en raw/[proyecto]/
```

```
  → Consultas: "estado de [Cliente]" → dashboard HTML
  → Reuniones de Meet → transcripción al correo asistente
  → Captura artículos con Web Clipper
  → Claude reporta en canal Slack al terminar cada tarea
  → Usuario aprueba o corrige outputs

17:00  → raw-processor sobre proyectos con ingresos nuevos
  → meet-processor si hay transcripciones pendientes

18:00  → /session-audit antes de cerrar
         Extrae preferencias y feedback de la sesión
  → flush-diario
         Logs → wiki, CRM y funnel actualizados
         HTML afectados regenerados
         Reporta en #brain-daily

Mensual (primer lunes):
  → skill-tuning analiza feedback acumulado
         Propone ajustes a los Super skills
```

PARTE 9 — MANTENIMIENTO DEL SISTEMA

9.1 Las capas y su lógica de mantenimiento

Capa	Contenido	Deterioro típico	Responsable
Identidad	claude.md, memory.md, entity-registry, skills	Se queda desactualizado	Claude actualiza, usuario revisa mensualmente
Trabajo activo	overviews, logs, entregables	Proyectos cerrados no archivados	Claude diario + usuario semanal
CRM y Funnel	05-CRM/, 08-Funnel/	Contactos sin actualizar	Claude en cada flush + usuario semanal
Memoria long-term	Pinecone + Biblioteca	Metadatos incompletos	Rutina mensual
Skills (Super)	feedback.md por skill	Patrones no incorporados al definicion.md	Rutina mensual `skill-tuning`

9.2 Qué hace el usuario manualmente

Diario (< 5 minutos):

- Depositar archivos físicos en `raw/`.
- Capturar artículos con Web Clipper.
- Verificar brief HTML de las 7:30 y confirmar prioridades.
- Aprobar o rechazar drafts en `outputs-ai/`.
- Ejecutar `/session-audit` al cerrar la sesión.

Semanal (< 30 minutos):

- Revisar `04-Inbox-Claude/` — asignar items huérfanos.
- Revisar `#brain-alertas` — atender pendientes.
- Revisar `08-Funnel/funnel.md` — confirmar estados.
- Archivar proyectos cerrados.

Mensual (< 1 hora):

- Revisar `memory.md` — corregir preferencias mal aprendidas.
- Revisar `claude.md` — actualizar instrucciones viejas.
- Revisar `entity-registry.md` — limpiar aliases obsoletos.
- Revisar reporte de `skill-tuning` y aprobar/rechazar ajustes propuestos.
- Auditar `raw/vector-inbox/` — confirmar docs sin ingestar.

Trimestral (< 2 horas):

- Correr cada skill manualmente y verificar el flujo.
- Limpiar `raw/procesados/` de archivos con más de 3 meses.
- Revisar `01-Biblioteca/` — identificar artículos para mover a Pinecone.
- Revisar `06-Archivo/` — confirmar documentación de proyectos cerrados.

9.3 Rutinas periódicas que corre Claude

Diaria — `daily-maintenance` (parte del flush-diario):

- Verifica `raw/[proyecto]/` con carpeta `procesados/`.
- Detecta archivos en `raw/` sin procesar por más de 3 días.
- Verifica que `outputs-ai/` del día tengan `estado:`.
- Actualiza `index.md`.
- Regenera HTML afectados.

Semanal — `vault-lint` (viernes al cierre): Genera `reporte_vault-lint_YYYY-MM-DD.html` con todos los hallazgos.

Mensual — `memory-review` (primer lunes):

1. Lee `memory.md` completo.
2. Detecta preferencias contradictorias o desactualizadas.
3. Propone versión limpia para aprobación.

Mensual — `skill-tuning` (primer lunes, junto con memory-review):

1. Lee `feedback.md` de todos los Super skills.
2. Detecta patrones recurrentes.
3. Propone ajustes al `definicion.md` de cada skill afectado.
4. Genera reporte HTML.

Mensual — `pinecone-audit` (primer lunes): Verifica metadatos completos en docs ingestados.

Nota sobre loops largos y context rot *(nuevo en v1.5)*

Cuando un skill procesa muchos archivos en una sola sesión (ej: `raw-processor` con un backlog grande, `library-compiler` con 30 artículos del Web Clipper), el contexto de Claude se llena rápido y la calidad de procesamiento empieza a degradarse. Esto se llama *context rot*.

Mitigación práctica: si vas a correr un loop de procesamiento largo, configurá un segundo loop en paralelo que ejecute `/clear` cada 30-60 minutos. El primer loop sigue procesando archivos, el segundo limpia contexto entre lotes. Funciona porque los skills bien definidos leen lo que necesitan en cada ejecución — no dependen de "recordar" lo que hicieron antes.

Esto solo aplica a procesamiento por lotes. Para skills agendados (`daily-brief`, `flush-diario`), cada ejecución arranca contexto limpio por diseño.

9.4 Señales de que el sistema necesita atención

- `raw/` con archivos sin procesar por más de una semana.
- `04-Inbox-Claude/` con más de 10 items.
- `#brain-alertas` con más de 5 mensajes no atendidos.
- CRM con contactos sin actualizar en proyectos activos.
- Funnel con filas sin next step.
- Claude respondiendo "no tengo información" en proyectos activos.
- `feedback.md` de un skill con más de 10 entradas sin pasar por `skill-tuning`.

PARTE 10 — PUESTA EN MARCHA PASO A PASO

10.1 Día 1 — Infraestructura base (2-3 horas)

Paso 1: GitHub

1. github.com → "New repository".
2. Nombre: `brain-os`. Visibilidad: Private.
3. Crear y copiar la URL.

Paso 2: GitHub Desktop

1. Descargar de desktop.github.com.
2. Clone repository → pegar URL.
3. Local path: `/Users/[tu-usuario]/Brain-OS/`.

Paso 3: Obsidian

1. Descargar de obsidian.md.
2. Open folder as vault → seleccionar Brain-OS/.

Paso 4: Estructura de carpetas

```
None
00-Sistema
00-Sistema/skills
00-Sistema/reportes
01-Biblioteca / frameworks / papers / regulacion / cursos
02-Proyectos
03-Daily-logs
04-Inbox-Claude
05-CRM
06-Archivo / proyectos-cerrados / contactos-inactivos
07-Finanzas / pagos-por-realizar / cuentas-por-cobrar
08-Funnel
raw / biblioteca/procesados / vector-inbox/procesados
```

Paso 5: Archivos base

En `00-Sistema/`, crear:

- `claude.md` — copiar template de Apéndice B y personalizar.
- `index.md` — vacío con encabezado.

- `memory.md` — vacío.
- `entity-registry.md` — copiar template.
- `projects.base` — copiar template.

En `08-Funnel/`, crear `funnel.md` con la tabla vacía.

Paso 6: Plugin Git

1. Settings → Community plugins → instalar Git.
2. Configuración: backup interval 1 min, Pull on startup ON, Push on backup ON.

Paso 7: Web Clipper

1. Chrome Web Store → "Obsidian Web Clipper".
2. Configurar: vault = Brain OS, folder = `raw/biblioteca`.

Paso 8: Cuenta del asistente

1. Crear `asistente.[tunombre]@gmail.com` o `asistente@[tudominio].com`.
2. Guardar credenciales.

Paso 9: Primer commit

1. GitHub Desktop → commit "Initial vault structure" → Push.

10.2 Día 1 continuación — Conectores Claude.ai (30 min)

Pasos 10-13

1. claude.ai → Settings → Connectors.
2. Conectar Gmail (cuenta personal): read-only.
3. Conectar Drive (shared drives): read-only.
4. Conectar Calendar: read + create con aprobación.
5. Conectar Gmail (asistente): read + archive.

10.3 Día 1 continuación — Pinecone (30 min)

Pasos 14-15

1. pinecone.io → Sign up free.
2. Create index: name = `brain-os`, dimensions = 1536, metric = cosine.
3. Guardar API key en gestor de contraseñas. Nunca dentro del vault.

10.4 Día 1 continuación — Slack (20 min)

Pasos 16-17

Crear los 6 canales:

- `#brain-archivos`
- `#brain-procesados`
- `#brain-transcripciones`
- `#brain-alertas`
- `#brain-daily`
- `#brain-dashboards` (nuevo en v1.4 — para notificar HTML generados)

10.5 Días 2-3 — Primer proyecto real (1-2 horas)

Pasos 18-20

1. Crear `02-Proyectos/[Cliente]/` con subcarpetas: insumos, entregables, financiero, outputs-ai.
2. Crear `overview.md` con template, `log.md` vacío.
3. Crear primer contacto en `05-CRM/`. Agregar al Entity Registry.
4. En Claude.ai: "Lee el claude.md y onboardá el proyecto [Cliente]."

10.6 Semana 1 — Cowork y primeros skills (2-3 horas)

Pasos 21-23

1. Instalar Claude Cowork. Apuntar a `/Users/[tu-usuario]/Brain-OS/`.
2. Crear primeros skills (Utility primero):
    - `estado-proyecto` (Super)
    - `draft-comunicacion` (Super)
    - `raw-processor` (Utility)
    - `meet-processor` (Utility)
3. Crear estructura `00-Sistema/skills/[nombre]/definicion.md` para cada uno.
4. Para los Super skills, crear también `feedback.md` vacío.
5. Primera ejecución manual de `vault-lint`.

10.7 Semanas 2-4 — Automatización (Fase 2)

Pasos 24-27

1. Scheduled tasks en Cowork:
    - `daily-brief` → 7:30 AM
    - `flush-diario` → 6:00 PM
    - `vault-lint` → viernes 5:00 PM
    - `skill-tuning` → primer lunes del mes 9:00 AM
2. Habilitar Obsidian CLI.
3. Crear `vector-ingest`. Procesar primeros docs de referencia.
4. Crear `library-compiler` (Super skill).

10.8 Mes 2 en adelante — Fase 3

Paso 28: Instalar Claude Code

1. Seguir las instrucciones en docs.anthropic.com.
2. Configurar el vault como proyecto de Claude Code.
3. Esto permite hooks de sesión y barreras técnicas que en Fase 1 y 2 son solo instrucciones en `claude.md`.

Paso 29: Configurar hooks de sesión

Brain OS usa cuatro hooks específicos en Fase 3. Cada uno transforma una regla del manual en comportamiento automático:

`session-start` hook Antes de la primera respuesta de cada sesión, Claude lee automáticamente:

1. `00-Sistema/claude.md` (constitución)
2. El daily log de ayer (`03-Daily-logs/YYYY-MM-DD.md`)
3. Items en `04-Inbox-Claude/sin-proyecto/` (cosas pendientes de asignar)

Esto reemplaza tener que recordarle a Claude que lea su contexto al inicio.

`pre-tool-use` hook — el más importante de Fase 3 Cuando Claude va a ejecutar una operación de escritura, el hook intercepta y verifica:

- Si la ruta destino contiene `07-Finanzas/` → bloquea automáticamente y pide confirmación explícita del usuario en esa sesión.
- Si la ruta destino contiene un drive personal del usuario → bloquea.

- Si la operación es borrar o sobreescribir sin que el usuario lo haya pedido explícitamente → bloquea.

Esto convierte la regla de territorio de Parte 2.4 (instrucción en `claude.md`) en una barrera técnica. La diferencia importa: una instrucción puede fallar si Claude se desvía; un hook no.

`post-tool-use` hook Después de ciertas operaciones, dispara notificaciones automáticas:

- Si se creó un archivo en `outputs-ai/` → notifica en `#brain-archivos`.
- Si se generó un HTML → notifica en `#brain-dashboards`.
- Si se movió un proyecto a `06-Archivo/` → notifica en `#brain-archivos`.

Esto elimina el riesgo de que Claude termine una tarea y olvide reportarla.

`session-end` hook Al cerrar la sesión, ejecuta automáticamente `/session-audit`. Esto extrae preferencias nuevas, las clasifica entre `memory.md`, `claude-project.md` o `feedback.md` del skill correspondiente, y muestra al usuario los hallazgos antes de guardar.

Paso 30: Cloud routines vs. local scheduled tasks

Migrar a Cloud routines los skills que no requieren CLI a Obsidian (`daily-brief`, `flush-diario`, `meet-processor`). Esto les permite correr aunque la máquina esté apagada. Los que requieren acceso directo al repo (`vault-lint`, `skill-tuning`, `memory-review`, `pinecone-audit`) se quedan como local scheduled tasks. Ver Parte 7.3 para el mapa completo.

10.9 Checklist de verificación

Fase 1 completa cuando:

- [ ] GitHub repo privado y vault local sincronizado.
- [ ] Estructura de carpetas completa (00-08 + raw/).
- [ ] Archivos base creados (claude.md, index.md, memory.md, entity-registry.md, projects.base, funnel.md).
- [ ] Conectores activos: Gmail (usuario y asistente), Drive, Calendar.
- [ ] 6 canales Slack creados.
- [ ] Pinecone index `brain-os` y API key guardada.
- [ ] Web Clipper instalado.
- [ ] Al menos un proyecto onbordeado con dashboard HTML inicial.
- [ ] Al menos un contacto en CRM y Entity Registry.

Fase 2 completa cuando:

- [ ] Cowork instalado y apuntando al vault.
- [ ] Estructura `00-Sistema/skills/[nombre]/` creada para cada skill.
- [ ] Super skills tienen su `feedback.md` activo.
- [ ] `daily-brief`, `flush-diario`, `vault-lint`, `skill-tuning` como scheduled tasks.
- [ ] Obsidian CLI habilitado.
- [ ] `vector-ingest` con primeros documentos en Pinecone.
- [ ] `library-compiler` probado.

Fase 3 completa cuando:

- [ ] Claude Code instalado y configurado con el vault.
- [ ] `session-start` hook lee claude.md y daily log de ayer automáticamente.
- [ ] `pre-tool-use` hook bloquea escritura en 07-Finanzas/ sin instrucción explícita.
- [ ] `post-tool-use` hook notifica creación de archivos en Slack correspondiente.
- [ ] `session-end` hook ejecuta `/session-audit` automáticamente.
- [ ] Cloud routines configuradas para skills sin dependencia de CLI (daily-brief, flush-diario, meet-processor).
- [ ] Local scheduled tasks confirmadas para skills con dependencia de CLI (vault-lint, skill-tuning, memory-review, pinecone-audit).

PARTE 11 — INSTRUCCIONES PARA CLAUDE AL RECIBIR ESTE DOCUMENTO

Modo operativo

- Estamos en modo operativo — no expliques qué es Obsidian o por qué usamos GitHub.
- Conocés la arquitectura, las herramientas y los flujos.
- Tu trabajo es producir outputs, no conversación.

Stacking de instrucciones

Al trabajar en cualquier tarea, leé en este orden — lo más específico gana:

1. `00-Sistema/claude.md` — constitución global
2. `02-Proyectos/[X]/claude-project.md` — preferencias del proyecto (si existe)
3. `02-Proyectos/[X]/overview.md` — contexto
4. `02-Proyectos/[X]/log.md` — entradas recientes
5. `00-Sistema/memory.md` — preferencias globales aprendidas
6. `00-Sistema/skills/[skill]/feedback.md` — ajustes específicos al skill activo

Markdown vs. HTML

- Fuente de verdad editable → Markdown.
- Vista derivada para leer/compartir → HTML autocontenido.

Genero HTML para: dashboards de proyecto, briefs diarios, reportes de lint, paneles de pendientes, timelines, cierres de proyecto.

Convención de nombres: `[tipo]_[descripción]_[ámbito].html`

Cuatro mecanismos de ingesta, ningún otro

1. Archivos físicos — `raw/[proyecto]/` o `raw/vector-inbox/`.
2. Gmail y Drive — vía MCP cuando se me pide.
3. Artículos web — Web Clipper → `raw/biblioteca/`.
4. Correo del asistente — transcripciones Meet y docs pasivos.

Reglas de memoria

- Ciclo de vida activo → Obsidian.
- Corpus terminado → Pinecone.
- Nunca duplico: si va a Pinecone, en Obsidian solo el índice.

Feedback loop por skill

Cuando el usuario corrige un output de un Super skill:

- Si la corrección es general → registro en `00-Sistema/memory.md`.
- Si es específica al proyecto → registro en `claude-project.md` del proyecto.
- Si es específica al skill → registro en `00-Sistema/skills/[skill]/feedback.md`.

La rutina mensual `skill-tuning` analiza estos feedbacks y propone ajustes estructurales.

Lo que nunca hago

- Acceder a `07-Finanzas/` sin instrucción explícita en esta sesión.
- Crear una entidad sin verificar el Entity Registry.
- Acceder a drives personales del usuario.
- Repetir información que ya está en el vault.
- Generar texto de chat cuando corresponde un archivo.
- Empezar respuestas con "¡Claro!" o "Por supuesto".
- Dejar archivos en `raw/` sin mover a `procesados/`.
- Enviar correos por la cuenta del asistente sin aprobación explícita.
- Reportar la misma acción en múltiples canales de Slack.

Personalización por usuario

```
None
## Quién soy
Nombre: [Nombre del usuario]
Idioma principal: [español / inglés / otro]
Zona horaria: [UTC-X]
Dominio: [consultoría / academia / derecho / etc.]

## Proyectos activos al día de hoy
1. [Cliente] — [fase] — [próxima fecha]

## Preferencias específicas
- [Ejemplo: resúmenes ejecutivos máximo 1 página]
```

```
## Contexto que no está en el vault
- [Sensibilidades de clientes específicos]
```

APÉNDICE A — Glosario del sistema

Término	Significado
vault	El repositorio Obsidian en `/Brain-OS/`
raw	Información sin procesar en la Capa A
wiki	Información compilada por Claude en Obsidian
compilar	Decirle a Claude "procesá esto"
flush	Promover daily logs a la wiki
onboarding	Incorporar un proyecto nuevo
skill	Workflow reusable en Cowork
Utility skill	Skill de tarea única, sin memoria entre ejecuciones
Super skill	Skill con feedback loop y memoria propia (`feedback.md`)
Project Operating Manual	`claude-project.md` con preferencias del proyecto
stacking de instrucciones	Lectura en cascada: global → proyecto → skill
session-audit	Comando de cierre que extrae preferencias de la sesión
skill-tuning	Rutina mensual que ajusta Super skills basado en feedback
output	Archivo listo en la carpeta correcta
estado	Campo YAML obligatorio en outputs Markdown
Entity Registry	Registro canónico de nombres propios
claim tipado	Callout: Source, Analysis, Unverified, Gap

Término	Significado
prefijo parseable	`## [YYYY-MM-DD] tipo | título` en logs
territorio Claude	Carpetas 00-06 y 08
territorio usuario	Carpeta 07-Finanzas
correo del asistente	Canal de ingesta pasiva — solo recibe
vector-inbox	`raw/vector-inbox/` — docs que van a Pinecone
Pinecone	Base de datos vectorial cloud
CRM	`05-CRM/` — un archivo Markdown por contacto
funnel	`08-Funnel/funnel.md` — tabla de oportunidades
HTML de doble vía	Output que el usuario lee como dashboard y Claude consulta como contexto

APÉNDICE B — Templates listos para copiar

Template: `claude.md` inicial

```
None
# Claude en Brain OS — Instrucciones maestras
Versión: [fecha]

## Mi identidad
Soy el colaborador digital del usuario. Opero con autonomía delegada.

## Mis herramientas
- Gmail MCP (usuario), Gmail MCP (asistente)
- Google Drive MCP (shared drives), Calendar MCP
- Obsidian CLI, GitHub, Pinecone
- Slack (canales por tipo)

## Territorio de acceso
- Carpetas 00-06 y 08: accedo libremente
- 07-Finanzas/: NO accedo salvo instrucción explícita

## Stacking de instrucciones
1. Este claude.md (constitución)
2. claude-project.md del proyecto (si existe)
3. overview.md y log.md del proyecto
4. memory.md (preferencias globales)
5. feedback.md del skill activo

## Markdown vs. HTML
- Fuente de verdad editable → Markdown
- Vista derivada para leer/compartir → HTML
- Convención HTML: [tipo]_[descripción]_[ámbito].html

## Entity Registry
Verifico entity-registry.md antes de crear entidades nuevas.

## Formato de log
Siempre: ## [YYYY-MM-DD] tipo | Descripción

## Feedback por skill
Correcciones a Super skills van a feedback.md del skill, no a memory.md.
```

Template: `claude-project.md`

```
None
---
proyecto: [Nombre]
creado: YYYY-MM-DD
ultima-actualizacion: YYYY-MM-DD
---

## Preferencias específicas
-

## Tono y estilo
[Consideraciones particulares]

## Convenciones internas del cliente
-

## Stakeholders y dinámicas
-
```

Template: `feedback.md` de Super skill

```
None
# Feedback acumulado — [nombre-skill]
Última revisión: YYYY-MM-DD

## [YYYY-MM-DD] ajuste-pedido
**Output original:** [resumen o link]
**Qué pidió corregir el usuario:** [descripción]
**Patrón detectado:** [si aplica]
```

Template: contacto CRM

```
None
---
nombre: [Nombre completo]
empresa: [Organización]
rol: [Cargo]
correo: [email]
relacion: cliente | prospecto | aliado | institucional | personal
estado: activo | inactivo | archivado
proyectos-asociados:
  - "[[02-Proyectos/Nombre-Proyecto]]"
```

```
ultima-actualizacion: YYYY-MM-DD
---

## Notas de relación

### [YYYY-MM-DD]
[Entrada más reciente primero]
```

Template: notificación Slack

```
None
[TIPO] [Proyecto/Área] — [descripción]
Archivos: [lista o link a HTML]
Pendientes detectados: [lista | ninguno]
```

Tipos: `[PROCESADO]` `[TRANSCRIPCIÓN]` `[ARCHIVADO]` `[ALERTA]` `[BRIEF]` `[FLUSH]` `[VAULT-LINT]` `[CRM]` `[FUNNEL]` `[DASHBOARD]`